

Computer Vision and Deep Learning

Lab 01: Introduction

About me

Francesco Dibitonto

- **Master's degree in Computer Engineering**, Polytechnic University of Turin (2021)
- **Research Assistant**, Polytechnic University of Turin (2021)
- **Research Scientist**, EVS – Embedded Vision Systems (2022-now)
- **PhD Student in Computer Science**, University of Verona (2024-now)

An initial FAQ

- *What will we learn in this lab?*
 - Implementations of various algorithms seen during the lectures in *Python*
- *What can we do if we have doubts regarding the arguments treated during the labs?*
 - You can contact me anytime via mail at francesco.dibitonto@univr.it

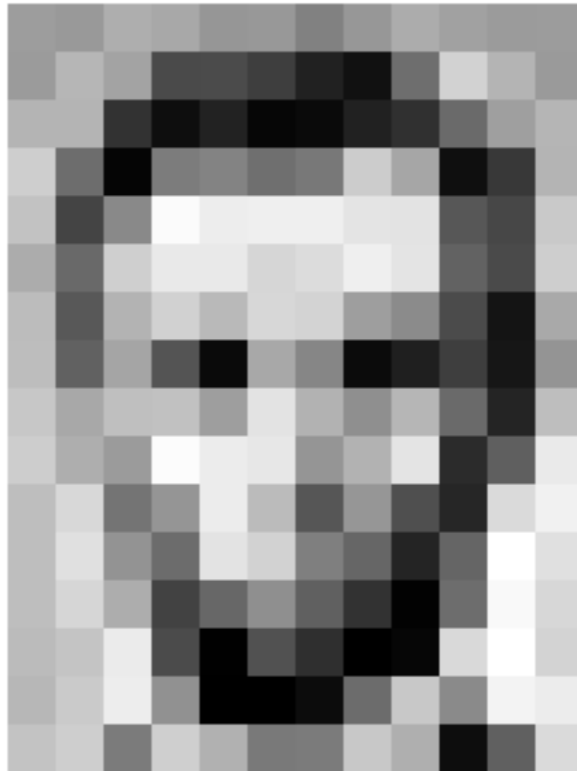
An initial FAQ (cont'd)

- *How are the lab lectures structured?*
 - Revision of the code from the previous labs
 - Brief summary of the theory
 - Coding exercises
- You will work on the code during the lab by yourself or with your colleagues and we will then see possible solutions together

What is Computer Vision?

- ***Computer Vision (CV)***: The field of computer science that focuses on enabling computers to identify and process objects in images and videos in the same way that humans do
- Starting from a set of data, we can try to understand building artificial systems that process, perceive, and reason about visual data

Example



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	105	159	181
206	109	5	124	131	111	120	204	165	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	105	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	105	159	181
206	109	5	124	131	111	120	204	165	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	105	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Pixel data diagram. At left, our image of Lincoln; at center, the pixels labeled with numbers from 0–255, representing their brightness; and at right, these numbers by themselves.

More recent examples

- 2D human pose (body + face + hand)
 - <https://openmmlab-open.oss-cn-shanghai.aliyuncs.com/www/video/MMPose2D.webm>
- Real-time semantic segmentation
 - https://www.youtube.com/watch?v=qWl9idsCuLQ&ab_channel=HengshuangZhao
- Multi-Object Tracking
 - <https://motchallenge.net/method/MOT=4808&chl=13>

CV and Deep Learning

- The development of deep learning technologies has enabled the creation of more accurate and complex computer vision models
- As these technologies increase, the incorporation of computer vision applications is becoming more useful

CV and Python

- Python is a high-level, multi-purpose, cross-platform programming language
- Can be used in an object oriented or functional manner
- In recent years, it has been widely used for scientific computing, especially for Machine Learning

Python

- Weird stuff:
 - Dynamic typing
 - Indentation is very important
 - Multiple differences between version 2.7 and 3+
- IDEs:
 - VS Code (my personal favourite)
 - PyCharm (specific IDE)
 - Bash command `python` opens an interactive shell

(*) <https://docs.python.org/3/tutorial/>

Conda

- Install Anaconda:
 - <https://www.anaconda.com/products/individual#Downloads>
 - Create a virtual env:
 - `$ conda create -n <name> python=3.8`
 - Activating and deactivating the env:
 - `$ conda activate <name>`
 - `$ conda deactivate`
- (*) <https://conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html>

Python package manager

- Python has an internal system for downloading, storing, and resolving packages. The most common packages (open source, third-party) are available in a repository called the *Python Package Index*¹ (PyPI):
 - `$ pip install numpy`
 - `$ pip install scipy==1.2.1 # install specific version`
 - `$ pip install --user Pillow # install only for current user`
- A `requirements.txt` file is handy for saving all the packages on which the project depends:
 - `$ pip install -r requirements.txt`

(¹) <https://packaging.python.org/en/latest/tutorials/installing-packages/>

Importing packages

- Python is so popular because of its packages and libraries. Here's an import of *numpy* in the shell:
 - `>> import numpy as np`
- **N.B:** pip installs packages system-wide (globally). It may not always be the best solution (compatibility issues between versions, need to be superuser etc.)

Python: Lists, vectors and matrices

- `>> a = 5` `# scalar`
- `>> b = [2, 3]` `# list, len(b): 2`
- `>> c = [[1,2],[3,4]]` `# list of lists. len(c): 2`

- `>> import numpy as np`
- `>> d = np.array([0])` `# scalar`
- `>> e = np.array([2, 3])` `# vector`
- `>> F = np.array([[1,2],[3,4]])` `# matrix`

Extracting data from matrices

- `>> a = [10,20,30,40]`
- `>> b = a[0:2]` `# b = [10,20]`
- `>> A = np.array([[10,20,30],[40,50,60]])`
- `>> B = A[:2,:2]`

Matrix operations

- Crucial for scientific computing and also Deep Learning
 - `>> B = A + A`
 - `>> C = A ** 2` # power of 2
 - `>> C = A * A` # element-wise
 - `>> C = np.dot(A,A)`
 - `>> D = np.divide(A,A)` # element-wise
 - `>> D = np.dot(A, np.linalg.inv(A))`
 - `>> np.sum(A)`
 - `>> np.sum(A, axis=0)` # over rows
 - `>> np.mean(A, axis=1)` # over cols

Running Python code

- Python scripts are simple text files with a `.py` extension
- To execute a script: `python <name>.py`
- Debugging: depends on the IDE
- A lot of useful information can be found in the python cheatsheet in the material

Jupyter Notebook

- Unique set-up for programming: open-source web-based application that allows you to create and share documents that contain live code, equations, visualizations and narrative text
 - <https://jupyter.org/>
- Very useful for sharing and interactive output functionalities
- Files end with a `.ipynb` extension

(*) <https://medium.com/getting-started-with-jupyter-notebook-for-python>

Google Colab

- Free platform for developing Python code on cloud based on Jupyter Notebooks provided by Google
- PROs:
 - Most scientific Python libraries already installed
 - Access to hardware accelerated GPUs and TPUs

(*) <https://towardsdatascience.com/getting-started-with-google-colab-f2fff97f594c>

Exercises